

CSC1003 Practice 7

Recursion

Brief Review

- What is recursion?

Briefly, it means “a function calls itself”. (Sometimes it can be implemented through “stack”, which you will learn later.)

Think of you solve a problem by breaking it down into smaller, identical problems. You keep breaking it down until you reach a version of the problem that is so simple it can be solved directly.

Brief Review

Note (about recursion):

- Must have a base case. (termination condition)
- Move toward to the base case.
(Or it will result in infinite recursion.)
- Pay attention to handling boundary conditions.
(Ensure that you have handled all possible situations.)

Brief Review

```
public static String convert(int N)
{
    if (N == 1) return "1";
    return convert(N/2) + (N % 2);
}
```

To compute a function of a
positive integer N

- Base case: 'N == 1'.
- Move toward to the base case by dividing 2 each time.
- Cannot handle the situation when $N = 0$. But N is positive, so it does not matter.

Practice A (Function)

Read a positive integer x from the input, print true if x is a palindrome; otherwise, print false. A palindrome is an integer that reads the same in original order (from left to right) and in reverse order (from right to left). For example, 121 is a palindrome, but 123 is not.

Input:

```
1123211
```

Output:

```
true
```

https://oj.cuhk.edu.cn/d/csc1003_2025_fall/p/P1012

Practice B

You are given two integers, a base a and an exponent b (which can be very large).

Your task is to calculate the value of $a^b \bmod (10^9 + 7)$ efficiently.

Sample input 1: 3 4

Sample output 1: 81

Sample input 2: 10 10

Sample output 2: 999999937

Sample input 3: 192 987654321

Sample output 3: 679747366

Hint:

$$\text{If } b \text{ is even: } a^b = (a^{(b/2)})^2$$

$$\text{If } b \text{ is odd: } a^b = a \times (a^{((b-1)/2)})^2$$

Practice B

Simple implementation of recursive and loop versions:

```
private static int power(int a, int b, int mod) {  
    int answer = 1;  
    for(int i = 1; i <= b; i++) {  
        answer = (int)((long) answer * a % mod);  
    }  
    return answer;  
}
```

```
private static int power(int a, int b, int mod) {  
    if (b == 0) return 1;  
    return (int)((long) a * power(a, b - 1, mod) % mod);  
}
```

Practice B

Looking at the hint, we can implement a recursive solution that halves the exponent b with each step. This approach is significantly more efficient than the simple implementation. (with time complexity of $O(\log_2(b))$ compared to $O(b)$)

```
private static int power(int a, int b, int mod) {
    if (b == 0) return 1;
    if (b % 2 == 0) {
        int half = power(a, b / 2, mod);
        return (int)((long) half * half % mod);
        // We can easily calculate squares
        // and luckily  $a^{(b/2)}$  is a  $b$ -smaller problem
    }
    else {
        int half = power(a, b / 2, mod);
        return (int)((long) half * half % mod * a % mod);
    }
}
```

```
10 10
Calculating 10^10
b is even
Calculating 10^5
b is odd
Calculating 10^2
b is even
Calculating 10^1
b is odd
Calculating 10^0
for calculating 10^1, half=1
for calculating 10^2, half=10
for calculating 10^5, half=100
for calculating 10^10, half=100000
999999937
```

Practice B

If you are interested, you can try to change this recursion to a loop. (Hint: Consider binary decomposition of b)

Q&A